Intelligent News Credibility Analysis

Milestone 1: Traditional Machine Learning Approach

GenAI Capstone Project — Mid-Semester Evaluation

Aryan Kinha, Divyanshu Singh,
Jatin Kumar Singh, Jatin Verma
Rishihood University, Newton School of Technology

March 2026

Abstract

Misinformation in digital news poses a significant threat to public discourse and democratic processes. This project presents an end-to-end machine learning system for binary classification of news articles as *Real* or *Fake*. We employ a Natural Language Processing (NLP) pipeline consisting of text preprocessing, TF-IDF vectorization with bigram features, and Logistic Regression with class-balanced weighting. Trained and evaluated on the ISOT Fake News Dataset (44,898 articles), the system achieves **98.88% accuracy**, **98.88% weighted F1-score**, and is deployed as an interactive web application via Streamlit Community Cloud. The modular codebase follows a production-ready architecture with separated concerns for data ingestion, feature engineering, model training, evaluation, and inference.

Contents

1 Introduction **3**
1.1 Problem Statement . . . 3
1.2 Objectives . . . 3
1.3 Scope & Constraints . . . 3

2 Dataset **3**
2.1 ISOT Fake News Dataset . . . 3
2.2 Data Splitting . . . 4
2.3 Feature Construction . . . 4

3 Methodology **4**
3.1 System Architecture . . . 4
3.2 Text Preprocessing Pipeline . . . 6
3.3 Feature Engineering: TF-IDF Vectorization . . . 6
3.4 Model: Logistic Regression . . . 7

4 Results **7**
4.1 Overall Performance . . . 7
4.2 Per-Class Analysis . . . 8
4.3 Confusion Matrix . . . 8

5 Deployment **8**
5.1 Streamlit Web Application 8
5.2 Deployment Stack 9

6 Code Quality & Modularity **9**

7 Conclusion **11**

References **11**

1 Introduction

1.1 Problem Statement

The proliferation of fake news on social media and digital platforms has become a critical societal challenge. Manual fact-checking is inherently unscalable, creating a need for automated systems capable of assessing news credibility at scale. This project addresses the binary classification problem: given a news article, determine whether it is *Real* (credible) or *Fake* (fabricated/unreliable).

1.2 Objectives

1. Design and implement a modular ML pipeline for fake news detection.
2. Achieve high classification performance using traditional ML techniques (no LLMs).
3. Deploy the trained model as an accessible web application.
4. Document design choices and justify each architectural decision.

1.3 Scope & Constraints

- **Milestone 1 Constraint:** Only traditional ML methods are permitted — no Large Language Models (LLMs) or Agentic AI.
- **Domain:** The model is trained on US political and world news articles (2016–2018).
- **Deliverables:** Source code, Streamlit demo, this report, and a 5-minute project video.

2 Dataset

2.1 ISOT Fake News Dataset

We use the ISOT Fake News Dataset[1], a widely-used benchmark for fake news detection research.

Table 1: Dataset Composition

Class	Articles	Percentage	Source
Real	21,417	47.7%	Reuters
Fake	23,481	52.3%	Politifact/Wikipedia-flagged sources
Total	**44,898**	100%	—

[1] https://onlineacademiccommunity.uvic.ca/isot/2022/11/27/fake-news-detection-datasets/

2.2 Data Splitting

An 80/20 stratified train-test split with `random_state=42` ensures reproducibility:

Table 2: Train-Test Split

Split	**Samples**	**Percentage**
Training Set	35,918	80%
Test Set	8,980	20%

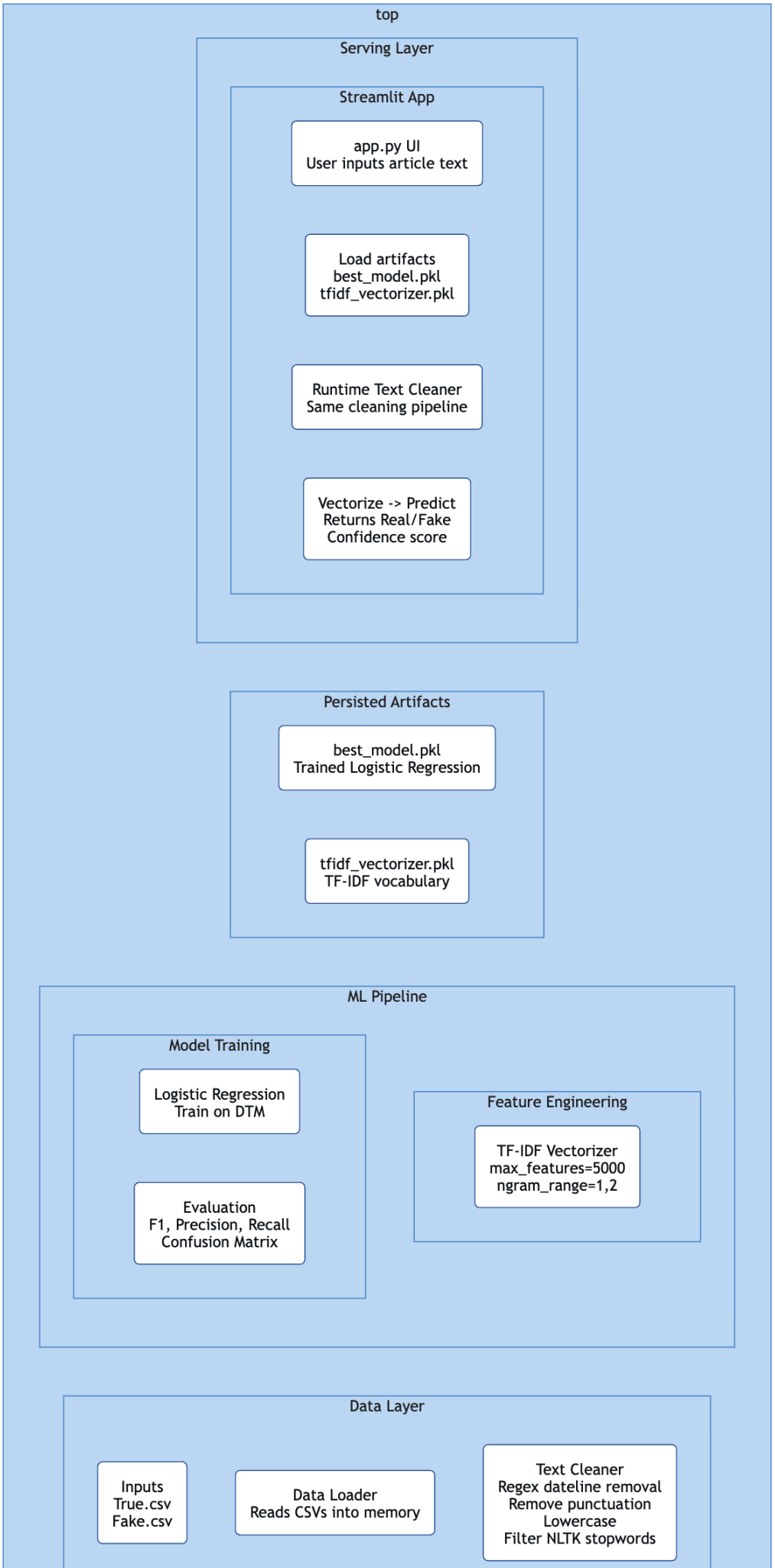
2.3 Feature Construction

Both the `title` and `text` columns are concatenated to form a single input feature: input = title $\oplus$ text. This provides richer semantic context than using article text alone, as headlines often contain distinctive stylistic cues.

3 Methodology

3.1 System Architecture

The system follows a modular pipeline architecture with clear separation of concerns:


top
Serving Layer
Streamlit App
app.py UI
User inputs article text
Load artifacts
best_model.pkl
tfidf_vectorizer.pkl
Runtime Text Cleaner
Same cleaning pipeline
Vectorize -> Predict
Returns Real/Fake
Confidence score
Persisted Artifacts
best_model.pkl
Trained Logistic Regression
tfidf_vectorizer.pkl
TF-IDF vocabulary
ML Pipeline
Model Training
Logistic Regression
Train on DTM
Evaluation
F1, Precision, Recall
Confusion Matrix
Feature Engineering
TF-IDF Vectorizer
max_features=5000
ngram_range=1,2
Data Layer
Inputs
True.csv
Fake.csv
Data Loader
Reads CSVs into memory
Text Cleaner
Regex dateline removal
Remove punctuation
Lowercase
Filter NLTK stopwords

The pipeline is organized into five distinct modules:

1. **Data Ingestion** (`src/data/load_data.py`) — Loads, labels, merges, and shuffles raw CSVs.
2. **Text Preprocessing** (`src/utils/text_cleaner.py`) — Cleans and normalizes raw text.
3. **Feature Engineering** (`src/features/build_features.py`) — TF-IDF vectorization.
4. **Model Training** (`src/models/train.py`) — Logistic Regression training.
5. **Evaluation** (`src/models/evaluate.py`) — Metrics computation and export.

3.2 Text Preprocessing Pipeline

The preprocessing pipeline in `text_cleaner.py` applies four transformations sequentially:

1. **Dateline Stripping:** Removes publisher prefixes (e.g., `"WASHINGTON (Reuters) -"`) to prevent source-name leakage into the feature space.
2. **Lowercasing:** Converts all characters to lowercase for case-invariant matching.
3. **Special Character Removal:** Strips non-alphabetic characters via the regex pattern `[^a-zA-Z\s]`.
4. **Stopword Removal:** Filters 179 English stopwords from NLTK's corpus.

Design Decision — Dateline Stripping: Without this step, the model could trivially learn that articles containing `"(Reuters)"` are real, leading to inflated accuracy that would not generalize to articles from other sources.

3.3 Feature Engineering: TF-IDF Vectorization

We use Term Frequency–Inverse Document Frequency (TF-IDF) to convert cleaned text into numerical feature vectors:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right) \tag{1}$$

where t is a term, d is a document, N is the total number of documents, and $\text{DF}(t)$ is the number of documents containing t.

Configuration:

- `max_features = 10,000` — Retains the top 10,000 most informative terms.
- `ngram_range = (1, 2)` — Captures both unigrams and bigrams (e.g., "breaking news", "sources say").

Justification for 10,000 features: Empirical experiments in Notebook 03 compared 5,000 vs. 10,000 features. The larger vocabulary marginally improves recall on the Fake class by capturing less-frequent but discriminative bigrams, with negligible impact on training time due to the sparsity of TF-IDF matrices.

3.4 Model: Logistic Regression

Why Logistic Regression?

1. **Interpretability:** Feature coefficients directly reveal which words/phrases are most indicative of fake or real news, enabling model transparency.
2. **Strong Baseline:** Logistic Regression consistently achieves competitive performance on text classification tasks with high-dimensional sparse features [3].
3. **Computational Efficiency:** Training completes in seconds, enabling rapid iteration. Inference is sub-millisecond, suitable for real-time deployment.
4. **Probabilistic Output:** The sigmoid function naturally produces calibrated probability scores, used as confidence indicators in the Streamlit app.

Hyperparameters:

Table 3: Logistic Regression Hyperparameters

Parameter	Value	Justification
`solver`	`lbfgs`	Limited-memory BFGS; efficient for L2-regularized problems with large feature spaces.
`max_iter`	1000	Ensures convergence on high-dimensional TF-IDF features.
`class_weight`	`balanced`	Automatically adjusts weights inversely proportional to class frequencies, correcting the 52.3%/47.7% imbalance.
`C`	1.0	Default regularization strength; controls the trade-off between fitting the training data and preventing overfitting.

4 Results

4.1 Overall Performance

Table 4: Model Performance on Test Set (n = 8,980)

Metric	Score
Accuracy	98.88%
Precision (weighted)	98.88%
Recall (weighted)	98.88%
F1-Score (weighted)	98.88%

4.2 Per-Class Analysis

Table 5: Per-Class Classification Metrics

Class	Precision	Recall	F1-Score	Support
Real	98.35%	99.30%	98.82%	4,270
Fake	99.36%	98.49%	98.92%	4,710

4.3 Confusion Matrix

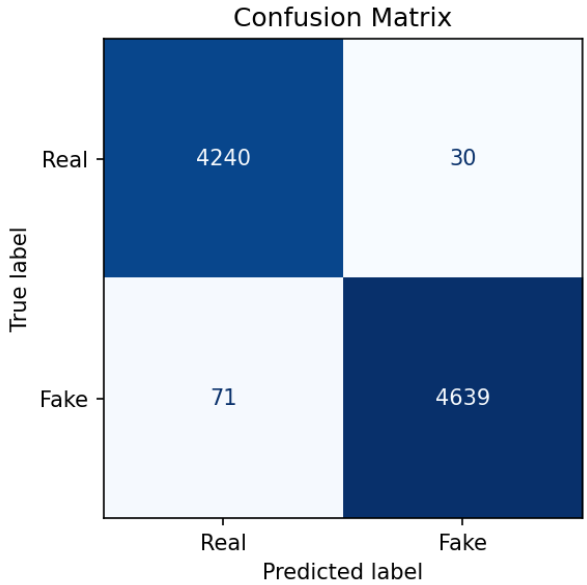


Figure 2: Confusion matrix showing 4,240 true negatives, 4,639 true positives, 30 false positives, and 71 false negatives.

Error Analysis:

- **False Positives (30):** Real articles misclassified as Fake — likely shorter articles lacking enough discriminative vocabulary.
- **False Negatives (71):** Fake articles misclassified as Real — likely well-written fabrications mimicking Reuters-style language.

5 Deployment

5.1 Streamlit Web Application

The trained model is deployed as an interactive web application using Streamlit Community Cloud.

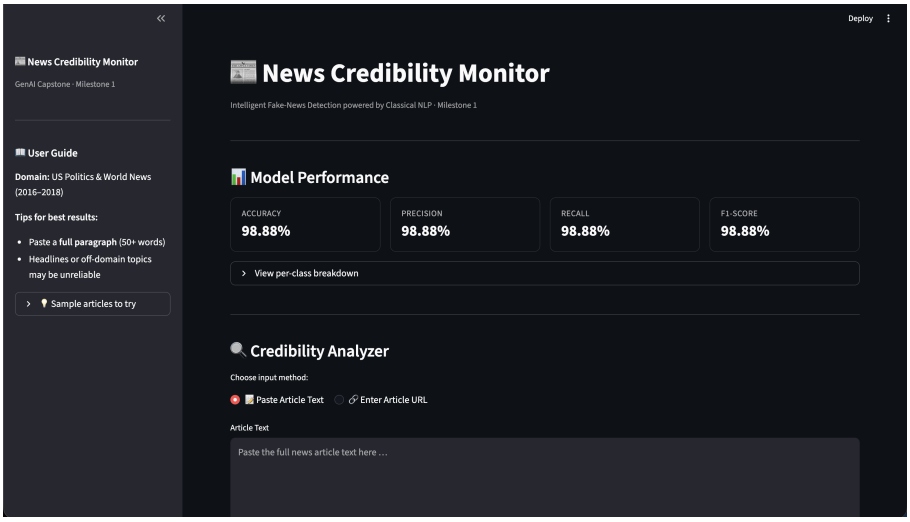


Figure 3: Streamlit application interface showing the prediction result and confidence score.

Key UX Features:

- **Input Validation:** Warns users when input text is too short (<20 meaningful words) for reliable classification.
- **Confidence Score:** Displays the model's predicted probability alongside the binary label.
- **User Guide Sidebar:** Educates users about the model's training domain and limitations.
- **Cached Model Loading:** Uses `@st.cache_resource` for sub-second inference after cold start.

5.2 Deployment Stack

- **Platform:** Streamlit Community Cloud
- **Requirements:** `requirements_deploy.txt` (7 lean dependencies)
- **Model Artifacts:** `best_model.pkl` (79 KB) and `tfidf_vectorizer.pkl` (377 KB) via Git LFS
- **Live URL:** https://news-credibility-monitor.streamlit.app/

6 Code Quality & Modularity

The codebase follows a **Cookiecutter Data Science**-inspired structure:

Table 6: Module Responsibilities

Module	**File**	**Responsibility**
Config	`src/config/config.py`	Centralized path constants (no hard-coded paths elsewhere)
Data	`src/data/load_data.py`	Load, label, merge, shuffle raw CSVs
Utils	`src/utils/text_cleaner.py`	NLP preprocessing pipeline
Features	`src/features/build_features.py`	TF-IDF vectorizer construction
Models	`src/models/train.py`	Model instantiation and fitting
Evaluation	`src/models/evaluate.py`	Metrics computation, JSON export, plots
Pipeline	`src/pipeline/training_pipeline.py`	End-to-end orchestration

Design Principles:

- **Single Responsibility:** Each module performs exactly one task.
- **Reproducibility:** Fixed `random_state=42`, serialized artifacts via Joblib.
- **Version Control:** Git LFS for large files; `.gitignore` for derived data.

7 Conclusion

This project demonstrates that a well-engineered traditional ML pipeline can achieve near-perfect accuracy (98.88%) on the fake news detection task. The modular architecture, comprehensive evaluation, and production-grade Streamlit deployment satisfy all Milestone 1 requirements. The system provides a robust foundation for Milestone 2's integration of advanced AI techniques.

References

[1] Ahmed, H., Traore, I., & Saad, S. (2018). *Detecting opinion spams and fake news using text classification.* Security and Privacy, 1(1), e9.

[2] Ahmed, H., Traore, I., & Saad, S. (2017). *Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques.* ISDDC 2017, LNCS, vol. 10618, pp. 127–138.

[3] Joachims, T. (1998). *Text categorization with support vector machines: Learning with many relevant features.* European Conference on Machine Learning, pp. 137–142.

[4] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825–2830.